

Trabajo Práctico Desarrollo Móvil: Android Studio, Kotlin y Jetpack Compose

Práctica Incremental

(v.1.2)

*Universidad Argentina de la Empresa
Facultad de Ingeniería y Cs. Exactas
Departamento de Tecnología Informática*

Esta guía propone una secuencia de **13 ejercicios incrementales**. No se presupone conocimiento previo de Kotlin. El objetivo no es solamente producir interfaces que “funcionen”, sino aprender a leer código Kotlin, navegar Android Studio, reconocer conceptos del SDK de Android, utilizar el emulador y comprender el modelo declarativo de Jetpack Compose.

Índice

1. Cómo trabajar con esta guía	3
1.1. Uso permitido de inteligencia artificial	3
1.2. Plantilla de prompt recomendada	3
2. Mapa de progresión sugerido	4
3. Ejercicio 1 — Primer proyecto, emulador y recorrido del entorno	5
4. Ejercicio 2 — Primeros pasos con funciones @Composable	6
5. Ejercicio 3 — Kotlin: val, var, tipos y plantillas de String	7
6. Ejercicio 4 — Funciones, parámetros y componentes reutilizables	8
7. Ejercicio 5 — Column, Row, Modifier y Material 3	9
8. Ejercicio 6 — Estado, botones y recomposición: contador	10
9. Ejercicio 7 — TextField, if/when y null safety	11
10. Ejercicio 8 — Calculadora pequeña, Logcat y debugger	12
11. Ejercicio 9 — data class, List y LazyColumn	13
12. Ejercicio 10 — Lista editable: tareas del estudiante	14
13. Ejercicio 11 — Recursos Android y configuración visual	15
14. Ejercicio 12 — Primer contacto con el Android SDK: Context e Intents	16
15. Ejercicio 13 — Miniaplicación integradora: organizador de cursada	17
16. Preguntas transversales para toda la práctica.	20
17. Registro opcional de uso de IA	20

18.Prompts que conviene evitar	20
19.Referencias oficiales recomendadas	21
20.Cierre	21

1. Cómo trabajar con esta guía

Cada ejercicio introduce pocos conceptos nuevos y reutiliza los anteriores. Salvo indicación contraria, se recomienda crear proyectos con una plantilla de actividad vacía preparada para Jetpack Compose y trabajar con Material 3.

Una modalidad posible para las entregas es conservar un proyecto por ejercicio. Otra alternativa es reutilizar un mismo proyecto y crear una pantalla o archivo Kotlin nuevo para cada actividad. Para un primer contacto suele ser más fácil trabajar con proyectos separados hasta que la estructura de Android Studio resulte familiar.

1.1. Uso permitido de inteligencia artificial

El equipo de cátedra permite utilizar asistentes de IA, pero el objetivo didáctico es que esta funcione como **tutor, revisor y herramienta de diagnóstico**, no como sustituto del estudiante.

Antes de copiar código generado por una IA, el estudiante debería poder contestar:

- ¿Qué hace cada función?
- ¿Qué valores pueden cambiar durante la ejecución?
- ¿Qué parte pertenece a Kotlin y qué parte pertenece a Android o Jetpack Compose?
- ¿Qué provocará una recomposición de la interfaz?
- ¿Qué error intenta solucionar el código?

Atención

Si una IA propone agregar bibliotecas, plugins o cambiar versiones de Gradle, Kotlin o Compose, **no realizar el cambio automáticamente**. Primero hay que pedirle que explique por qué es necesario y verificar si la solución puede realizarse con las dependencias que ya posee el proyecto.

1.2. Plantilla de prompt recomendada

Una estructura útil para casi todos los ejercicios es:

Estoy aprendiendo Android con Kotlin y Jetpack Compose y todavía soy principiante. No me des la solución completa. Explicame primero el concepto y dame una pista mínima para avanzar. Este es mi código: [código]. Este es el error o comportamiento que observo: [descripción]. Indicame qué debería revisar y por qué.

Cuando el programa ya funciona, puede pedirse:

Revisá este código como tutor de un curso introductorio. Señalá dos mejoras de legibilidad o diseño, pero no lo reescribas completo. Explicá cada sugerencia.

2. Mapa de progresión sugerido

Ej.	Tema principal	Nivel
1	Proyecto, AVD, ejecución y exploración del IDE	Inicial
2	Primeros composables y Preview	Inicial
3	Kotlin: variables, tipos y plantillas de String	Inicial
4	Funciones, parámetros y reutilización de UI	Inicial
5	Layouts, Modifier y Material 3	Inicial-medio
6	Estado, eventos y recomposición	Medio
7	Entrada de texto, condicionales y null safety	Medio
8	Funciones de cálculo, debugger y Logcat	Medio
9	data class, colecciones y LazyColumn	Medio
10	Lista editable y estado de una colección	Medio
11	Recursos Android, strings y configuración visual	Medio
12	Android SDK: Context e Intents	Medio
13	Miniaplicación integradora	Medio-alto

3. Ejercicio 1 — Primer proyecto, emulador y recorrido del entorno

Tiempo orientativo: 35–50 minutos

Consigna

Crear una aplicación llamada `MiPrimeraApp`. Antes de modificar código:

1. Recorrer la vista *Android* del panel Project.
2. Localizar `MainActivity.kt`.
3. Localizar `AndroidManifest.xml`.
4. Localizar el archivo Gradle del módulo `app`.
5. Abrir Device Manager.
6. Crear un AVD de teléfono si no existe ninguno.
7. Ejecutar la aplicación en el AVD.
8. Identificar en qué sector de Android Studio aparece el proceso de compilación.

Luego modificar únicamente el texto principal de la aplicación para que muestre:

`Hola Android - <nombre del estudiante>`

Volver a ejecutar la aplicación.

Preguntas

1. ¿Qué archivo contiene el código Kotlin principal?
2. ¿Qué diferencia observás entre hacer *Run* y abrir solamente un *Preview*?
3. ¿Qué es un AVD? ¿Es lo mismo que el emulador?
4. ¿Qué API de Android utiliza el dispositivo virtual creado?
5. ¿Qué ocurre si el AVD está apagado cuando se presiona Run?

Consejo de uso de IA

Prompt sugerido: “*Estoy usando Android Studio por primera vez. Explicame, sin darme código nuevo, qué función cumplen MainActivity.kt, AndroidManifest.xml, Gradle y el AVD. Usá una analogía sencilla y después una explicación técnica breve.*”

Criterio de finalización

La app debe ejecutarse en un AVD y mostrar el texto personalizado. El estudiante debe poder señalar físicamente en Android Studio los cuatro archivos o herramientas pedidos.

4. Ejercicio 2 — Primeros pasos con funciones @Composable

Tiempo orientativo: 30–45 minutos

Consigna

Crear una función composable llamada `Presentacion` que muestre tres textos:

- nombre;
- carrera;
- una tecnología que el estudiante quiera aprender.

Usar la función desde la interfaz principal y crear además una función marcada con `@Preview` para verla sin iniciar el emulador.

Código inicial orientativo:

```
@Composable
fun Presentacion() {
    // Agregar aquí tres Text(...)
}

@Preview(showBackground = true)
@Composable
fun PresentacionPreview() {
    // Invocar aquí Presentacion()
}
```

Experimentación

Realizar intencionalmente estas pruebas, una por vez:

1. eliminar temporalmente `@Composable`;
2. escribir mal el nombre de `Text`;
3. agregar un segundo `Text` sin usar todavía un layout;
4. observar los mensajes del editor y corregir el código.

Consejo de uso de IA

Prompt sugerido: “¿Qué significa `@Composable` en Jetpack Compose? Explicámelo para alguien que recién aprende funciones en Kotlin. Después explicame por qué una función `@Composable` puede llamar a `Text()`, pero una función Kotlin común no debería hacerlo directamente.”

Criterio de finalización

La función `Presentacion()` debe utilizarse tanto desde la pantalla de la app como desde un `@Preview`.

5. Ejercicio 3 — Kotlin: val, var, tipos y plantillas de String

Tiempo orientativo: 40–55 minutos

Consigna

Construir una pantalla “Ficha del estudiante”. Dentro de una función composable declarar datos como:

```
val nombre = "Ana"
val edad = 20
val promedio = 8.25
val cursaProgramacion = true
```

Mostrar los valores en la interfaz utilizando plantillas de String:

```
Text("Nombre: $nombre")
Text("Edad: $edad")
Text("Promedio: $promedio")
```

Agregar una variable adicional calculada, por ejemplo:

```
val anioProximo = edad + 1
```

y mostrar una frase que combine texto y expresión.

Parte B — Investigar el tipo

Crear algunas variables indicando el tipo explícitamente:

```
val materias: Int = 5
val ciudad: String = "Cordoba"
```

Compararlas con variables sin tipo explícito.

Responder:

1. ¿Cuándo usarías `val`?
2. ¿Cuándo sería necesario `var`?
3. ¿Qué significa que Kotlin “infiera” un tipo?
4. ¿Por qué `"20"` y `20` no representan lo mismo?

Consejo de uso de IA

Prompt sugerido: “*Dame cinco preguntas cortas para comprobar si entendí val, var, String, Int, Double y Boolean en Kotlin. No me muestres las respuestas hasta que yo conteste.*”

Criterio de finalización

La app debe presentar al menos cinco datos y una expresión calculada. El estudiante debe poder justificar el tipo de cada variable.

6. Ejercicio 4 — Funciones, parámetros y componentes reutilizables

Tiempo orientativo: 45–60 minutos

Objetivos

- Declarar funciones con parámetros.
- Comprender argumentos y tipos de retorno.
- Crear componibles reutilizables.
- Evitar duplicación simple de código.

Consigna

Crear un composable reutilizable:

```
@Composable
fun DatoEstudiante(etiqueta: String, valor: String) {
    // Mostrar etiqueta y valor
}
```

Usarlo varias veces:

```
DatoEstudiante("Nombre", "Ana")
DatoEstudiante("Carrera", "Sistemas")
DatoEstudiante("Año", "1")
```

Luego crear una función Kotlin no composable:

```
fun descripcionEdad(edad: Int): String {
    return "Edad: $edad años"
}
```

Mostrar su resultado con un Text.

Desafío breve del Profe Adrián

Modificar `descripcionEdad()` para que indique además si la persona es mayor de edad usando `if`.

Consejo de uso de IA

Prompt sugerido: “Tengo tres bloques de UI casi iguales en Compose. Antes de escribirme código, explicame cómo detectar duplicación y cómo decidir qué información debería transformarse en parámetros de una función.”

Criterio de finalización

Debe existir al menos un composable reutilizable con parámetros y una función Kotlin común que retorne un valor.

7. Ejercicio 5 — Column, Row, Modifier y Material 3

Tiempo orientativo: 50–70 minutos

Objetivos

- Organizar componentes con `Column` y `Row`.
- Introducir `Modifier`.
- Utilizar `padding`, `fillMaxWidth` y `Arrangement`.
- Explorar componentes de Material 3.

Consigna

Rediseñar la ficha de los ejercicios anteriores para que tenga:

- un título;
- datos organizados verticalmente;
- al menos una fila con dos elementos;
- espaciado exterior mediante `padding`;
- un `Button` todavía sin comportamiento relevante.

Ejemplo de esqueleto:

```
Column(  
    modifier = Modifier  
        .fillMaxWidth()  
        .padding(16.dp)  
) {  
    Text("Perfil")  
    // Otros componentes  
}
```

Experimento obligatorio del Profe Adrián

Intercambiar el orden de dos modificadores y observar si cambia el resultado. Por ejemplo, experimentar con combinaciones de `padding`, `fillMaxWidth` o fondo si se utiliza alguno.

Responder: ¿el orden de los `Modifier` puede afectar la interfaz?

Consejo de uso de IA

Prompt sugerido: “*Explicame Modifier en Jetpack Compose con tres ejemplos pequeños. No diseñes mi pantalla. Quiero entender por qué Modifier se encadena y por qué el orden de los modificadores puede cambiar el resultado.*”

Criterio de finalización

La pantalla debe utilizar `Column`, `Row`, al menos tres modificadores y un componente Material 3 adicional a `Text`.

8. Ejercicio 6 — Estado, botones y recomposición: contador

Tiempo orientativo: 50–65 minutos

Objetivos

- Comprender qué significa “estado” en una interfaz declarativa.
- Utilizar `remember` y estado mutable.
- Responder a eventos `onClick`.
- Observar la recomposición.

Consigna

Crear un contador con:

- texto con el valor actual;
- botón `+1`;
- botón `-1`;
- botón Reiniciar.

Esqueleto orientativo:

```
@Composable
fun Contador() {
    var contador by remember { mutableStateOf(0) }

    Text("Valor: $contador")

    Button(onClick = { contador++ }) {
        Text("+1")
    }
}
```

Agregar la condición de que el contador no pueda ser menor que cero.

Preguntas conceptuales

1. ¿Por qué un `val contador = 0` no alcanza para este ejercicio?
2. ¿Qué ocurre cuando cambia el estado?
3. ¿Qué parte de la interfaz depende de `contador`?
4. ¿Qué diferencia hay entre cambiar una variable y actualizar una variable de estado observada por Compose?

Consejo de uso de IA

Prompt sugerido: “No entiendo recomposición en Jetpack Compose. Explicame paso a paso qué sucede desde que presiono un Button hasta que el Text muestra el nuevo valor. Usá el ejemplo de un contador y distinguí evento, cambio de estado y recomposición.”

Criterio de finalización

Los tres botones deben funcionar y el valor nunca debe quedar debajo de cero.

9. Ejercicio 7 — TextField, if/when y null safety

Tiempo orientativo: 60–80 minutos

Objetivos

- Leer datos ingresados por el usuario.
- Usar `TextField` u `OutlinedTextField`.
- Convertir texto a número mediante `toIntOrNull()`.
- Trabajar con `null`.
- Aplicar condiciones con `if` o `when`.

Consigna

Crear un “Clasificador de edad”.

La pantalla tendrá:

- un campo para ingresar nombre;
- un campo para ingresar edad;
- un botón **Evaluar**;
- un texto con el resultado.

Comportamiento esperado:

- si la edad no puede convertirse a número, mostrar un mensaje de error;
- si es menor que 0, indicar que el dato no es válido;
- si es menor que 18, mostrar “menor de edad”;
- en otro caso, mostrar “mayor de edad”.

Investigar este patrón:

```
val edadNumero = edadTexto.toIntOrNull()

if (edadNumero == null) {
    // El texto no representa un Int valido
}
```

Desafío

Reemplazar la cadena de `if` por una expresión `when`.

Consejo de uso de IA

Prompt sugerido: “*Explicame qué devuelve toIntOrNull() en Kotlin y por qué su tipo contiene ?. No resuelvas mi ejercicio. Dame tres ejemplos de entradas y decime qué valor produciría cada una.*”

Criterio de finalización

El programa debe manejar correctamente una edad válida, texto no numérico, valor negativo y campo vacío sin cerrarse inesperadamente.

10. Ejercicio 8 — Calculadora pequeña, Logcat y debugger

Tiempo orientativo: 70–90 minutos

Consigna

Crear una calculadora de dos números que permita:

- sumar;
- restar;
- multiplicar.

La lógica debe estar en una función separada de la interfaz, por ejemplo:

```
fun calcular(a: Double, b: Double, operacion: String): Double {  
    return when (operacion) {  
        "sumar" -> a + b  
        "restar" -> a - b  
        "multiplicar" -> a * b  
        else -> 0.0  
    }  
}
```

No es obligatorio conservar exactamente esta firma si el estudiante propone una alternativa razonable.

Parte B — Debugger

1. Colocar un breakpoint dentro de la función de cálculo.
2. Ejecutar con Debug.
3. Ingresar dos valores.
4. Detenerse en el breakpoint.
5. Inspeccionar los valores de **a**, **b** y **operacion**.
6. Continuar la ejecución.

Parte C — Logcat

Agregar temporalmente un log al ejecutar una operación. Utilizar una etiqueta reconocible, por ejemplo "CALCULADORA", y filtrar Logcat por esa etiqueta.

Consejo de uso de IA

Prompt sugerido: "Mi app compila pero el resultado de esta función es incorrecto. No me des una función nueva. Indícame dónde pondrías un breakpoint, qué variables inspeccionarías y qué hipótesis comprobarías primero. Código: [pegar función]."

Criterio de finalización

La calculadora debe resolver las tres operaciones y el estudiante debe demostrar una ejecución detenida en un breakpoint, explicando los valores observados.

11. Ejercicio 9 — data class, List y LazyColumn

Tiempo orientativo: 70–90 minutos

Objetivos

- Introducir clases de datos en Kotlin.
- Crear y recorrer una colección.
- Mostrar listas con LazyColumn.
- Diferenciar los datos de su representación visual.

Consigna

Modelar una materia universitaria:

```
data class Materia(  
    val nombre: String,  
    val anio: Int,  
    val aprobada: Boolean  
)
```

Crear una lista con al menos cinco materias:

```
val materias = listOf(  
    Materia("Programacion I", 1, true),  
    Materia("Matematica", 1, false)  
    // ...  
)
```

Mostrarlas con una LazyColumn. Cada fila debe incluir:

- nombre;
- año;
- estado de aprobación.

Crear un composable `MateriaItem(materia: Materia)` para representar una sola fila.

Desafíos

1. Mostrar solamente las materias aprobadas utilizando `filter`.
2. Mostrar la cantidad total mediante `materias.size`.
3. Cambiar visualmente el texto según `aprobada`.

Consejo de uso de IA

Prompt sugerido: “*Explicame la diferencia entre data class, List y LazyColumn usando este ejercicio de materias. No generes la pantalla. Quiero entender qué parte representa datos de Kotlin y qué parte pertenece a Jetpack Compose.*”

Criterio de finalización

Debe visualizarse una lista de al menos cinco objetos de tipo `Materia`, utilizando un componente independiente para cada elemento.

12. Ejercicio 10 — Lista editable: tareas del estudiante

Tiempo orientativo: 80–110 minutos

Objetivos

- Manejar el estado de una colección.
- Agregar y quitar elementos.
- Combinar `TextField`, botones y `LazyColumn`.
- Empezar a pensar la UI como función del estado.

Consigna

Crear una aplicación de tareas de estudio.

Requisitos mínimos:

- campo para escribir una tarea;
- botón **Agregar**;
- lista de tareas;
- posibilidad de eliminar una tarea;
- no agregar cadenas vacías.

Antes de buscar la solución, dibujar en papel el estado mínimo de la aplicación. Identificar qué valores cambian: el texto actualmente escrito y la colección de tareas.

Desafíos

1. Mostrar la cantidad de tareas.

2. Agregar botón “Borrar todas”.
3. Permitir marcar una tarea como completada.

Consejo de uso de IA

Prompt sugerido: “Ayúdame a modelar el estado de una lista de tareas en Compose. No escribas la app. Primero preguntame qué datos necesito conservar. Después revisá mi propuesta de variables de estado y decime si sobra o falta alguna.”

Criterio de finalización

Debe ser posible agregar al menos tres tareas y eliminarlas sin reiniciar la aplicación.

13. Ejercicio 11 — Recursos Android y configuración visual

Tiempo orientativo: 60–80 minutos

Objetivos

- Comprender que no todo debe escribirse directamente en Kotlin.
- Utilizar recursos de texto.
- Acceder a recursos desde Compose.
- Explorar tema claro/oscuro y Preview.

Consigna

Tomar uno de los proyectos anteriores y mover los textos importantes a recursos de Android.

Por ejemplo, en `res/values/strings.xml`:

```
<resources>
  <string name="app_name">Mis Tareas</string>
  <string name="titulo_tareas">Tareas de estudio</string>
  <string name="agregar">Agregar</string>
</resources>
```

Luego utilizarlos desde Compose con `stringResource`.

Crear al menos dos previews: uno en configuración normal y otro en modo oscuro o con otra configuración visual disponible.

Preguntas

1. ¿Por qué Android separa recursos del código?
2. ¿Qué ventaja tiene esto para traducir una aplicación?
3. ¿Por qué Preview es útil aunque ya exista un emulador?

Consejo de uso de IA

Prompt sugerido: “*Explicame por qué Android usa resources y strings.xml si en Compose puedo escribir Text("Hola") directamente. Compará ambos enfoques pensando en mantenimiento e internacionalización.*”

Criterio de finalización

Al menos cuatro textos visibles deben provenir de recursos y la pantalla debe poseer más de un Preview útil.

14. Ejercicio 12 — Primer contacto con el Android SDK: Context e Intents

Tiempo orientativo: 70–95 minutos

Objetivos

- Distinguir Kotlin, Compose y APIs del Android SDK.
- Comprender de forma inicial qué representa un `Context`.
- Crear un `Intent`.
- Delegar una acción a otra aplicación del dispositivo.

Consigna

Crear una pantalla “Compartir recomendación” con un texto y un botón.

Al presionar el botón, abrir el selector de aplicaciones del sistema para compartir un mensaje de texto mediante un `Intent.ACTION_SEND`.

Investigar los siguientes elementos:

```
val context = LocalContext.current

val intent = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Estoy aprendiendo Android con Compose")
}
```

Luego iniciar la acción correspondiente desde el contexto, utilizando un chooser si corresponde.

Parte B

Agregar otro botón que abra una dirección web mediante un `Intent.ACTION_VIEW` y una URI.

Preguntas

Clasificar cada elemento como **Kotlin**, **Compose** o **Android SDK**: `val`, `fun`, `Button`, `@Composable`, `Intent`, `Context` y `Uri`.

Consejo de uso de IA

Prompt sugerido: “En este fragmento marcame qué cosas pertenecen al lenguaje Kotlin, cuáles a Jetpack Compose y cuáles al Android SDK. No cambies el código. Después explicame para qué sirve Context en este caso.”

Atención

Para este ejercicio no es necesario solicitar permisos peligrosos. Si una respuesta de IA propone modificar permisos sin explicar por qué, detenerse y revisar la propuesta antes de aplicarla.

Criterio de finalización

La aplicación debe poder abrir el selector para compartir texto y lanzar al menos una acción ACTION_VIEW apropiada.

15. Ejercicio 13 — Miniaplicación integradora: organizador de cursada

Tiempo orientativo: 2–3 horas, preferentemente dividido en etapas

Objetivos

- Integrar Kotlin básico, Compose, estado, listas y recursos.
- Diseñar una aplicación pequeña antes de programarla.
- Practicar depuración y lectura de errores.
- Utilizar IA de manera trazable y crítica.

Consigna general

Construir un “Organizador de cursada”.

Cada elemento representará una actividad académica:

```
data class Actividad(  
    val titulo: String,  
    val materia: String,  
    val prioridad: Int,  
    val completada: Boolean  
)
```

La aplicación debe permitir:

1. ingresar título;
2. ingresar o seleccionar materia;
3. asignar una prioridad simple;
4. agregar la actividad;
5. listar actividades con LazyColumn;

6. marcar una actividad como completada;
7. eliminar una actividad;
8. mostrar cuántas están pendientes.

Etapa 1 — Diseño

Antes de programar, entregar un pequeño esquema con los datos que necesita cada actividad, los estados que necesita la pantalla, los composables que podrían reutilizarse y los eventos posibles del usuario.

Etapa 2 — Implementación mínima

Primero implementar solamente una lista fija, una fila reutilizable y un contador de elementos. No avanzar hasta que esta versión compile y funcione.

Etapa 3 — Interacción

Agregar campos de entrada, alta de actividades, eliminación y cambio del estado `completada`.

Etapa 4 — Calidad

Agregar strings como recursos, validación de campos, al menos un Preview, nombres de funciones y variables claros, un breakpoint utilizado durante una prueba y al menos un mensaje de Logcat utilizado para diagnóstico.

Desafíos opcionales

1. filtrar por pendientes/completadas;
2. ordenar por prioridad;
3. agregar botón para compartir el resumen mediante un Intent;
4. mostrar un mensaje cuando no existen actividades;
5. crear diferentes previews con datos de ejemplo.

Consejo de uso de IA

Diseño: “Actuá como tutor. Revisá este modelo de datos y esta lista de estados para una app de actividades universitarias. No escribas código. Decime si alguna responsabilidad está mezclada.”

Error: “Este código produce el siguiente error. Explicame qué significa el mensaje, señalá la línea más probable y sugerime un único cambio pequeño para comprobar la hipótesis. No reescribas el archivo completo.”

Revisión: “Mi app ya funciona. Señalá tres lugares donde podría mejorar nombres, repetición o separación entre datos y UI. Priorizá mejoras apropiadas para un principiante.”

Criterio de finalización

La miniaplicación debe cumplir los ocho requisitos mínimos. Además, el estudiante debe poder explicar oralmente dónde se encuentra el estado, qué composables son reutilizables y qué parte del proyecto utiliza una API propia de Android.

16. Preguntas transversales para toda la práctica.

1. ¿Qué parte del código es Kotlin puro?
2. ¿Qué parte es Jetpack Compose?
3. ¿Qué parte depende del Android SDK?
4. ¿Qué datos son constantes y cuáles constituyen estado?
5. ¿Qué evento puede cambiar cada estado?
6. ¿Qué parte de la UI debería recomponerse cuando cambia?
7. ¿Qué función podría extraerse para evitar código repetido?
8. ¿El error que aparece es de sintaxis, tipos, compilación o ejecución?
9. ¿Podría observar el problema con un breakpoint?
10. ¿Podría observar el problema en Logcat?
11. ¿Es necesario ejecutar el emulador o alcanza con Preview?
12. ¿La solución propuesta por una IA utiliza conceptos que todavía no puedo explicar?

17. Registro opcional de uso de IA

Para la parte práctica de parciales y finales, podremos solicitar un registro muy breve de uso de IA:

1. **Problema:** ¿qué quería resolver?
2. **Prompt:** ¿qué se le pidió a la IA?
3. **Respuesta útil:** ¿qué concepto o pista aportó?
4. **Decisión propia:** ¿qué se modificó realmente?
5. **Verificación:** ¿cómo se comprobó que funcionaba?

El objetivo no es penalizar el uso de IA, sino hacer visible el proceso de razonamiento y fomentar que el estudiante pueda validar lo que recibe.

18. Prompts que conviene evitar

Ejemplos poco útiles para aprender:

- “Haceme el ejercicio 10.”
- “Generame una app de tareas en Compose.”
- “Corregí todo mi proyecto.”
- “Dame el código final.”
- “No funciona, volvé a iterar”

Versiones más productivas:

- “Mi botón no agrega elementos. Ayúdame a identificar qué estado debería cambiar.”
- “Explicame este error de tipos y dame una pista, no la solución completa.”
- “Compará List y una lista mutable para este caso y decime qué debería estudiar.”
- “Revisá esta función y haceme preguntas para comprobar si entiendo lo que hace.”

19. Referencias oficiales recomendadas

- Android Basics with Compose: <https://developer.android.com/courses/android-basics-compose/course>
- Introducción a Jetpack Compose: <https://developer.android.com/develop/ui/compose/documentation>
- Estado en Compose: <https://developer.android.com/develop/ui/compose/state>
- Listas en Compose: <https://developer.android.com/develop/ui/compose/lists>
- Preview de composables: <https://developer.android.com/develop/ui/compose/tooling/previews>
- Recursos en Compose: <https://developer.android.com/develop/ui/compose/resources>
- Ejecutar y depurar una app: <https://developer.android.com/studio/run>
- Debugger de Android Studio: <https://developer.android.com/studio/debug>
- Crear y administrar AVD: <https://developer.android.com/studio/run/managing-avds>
- Intents: <https://developer.android.com/guide/components/intents-filters>
- Sintaxis básica de Kotlin: <https://kotlinlang.org/docs/basic-syntax.html>
- Null safety de Kotlin: <https://kotlinlang.org/docs/null-safety.html>

20. Cierre

Al completar esta secuencia de forma conciente, se espera que puedan crear y ejecutar una aplicación sencilla, reconocer los archivos y herramientas principales del entorno, leer Kotlin elemental, construir interfaces declarativas con Compose, manejar estado y eventos, presentar colecciones, utilizar recursos del proyecto, realizar un primer uso del Android SDK y diagnosticar problemas básicos con Android Studio.

El objetivo no es memorizar APIs, sino desarrollar una primera estrategia de trabajo:

Cariños totales, Ale y Adrián **leer, modificar poco, ejecutar, observar, depurar, explicar y recién después ampliar la solución.**

Cariños totales, Ale y Adrián.